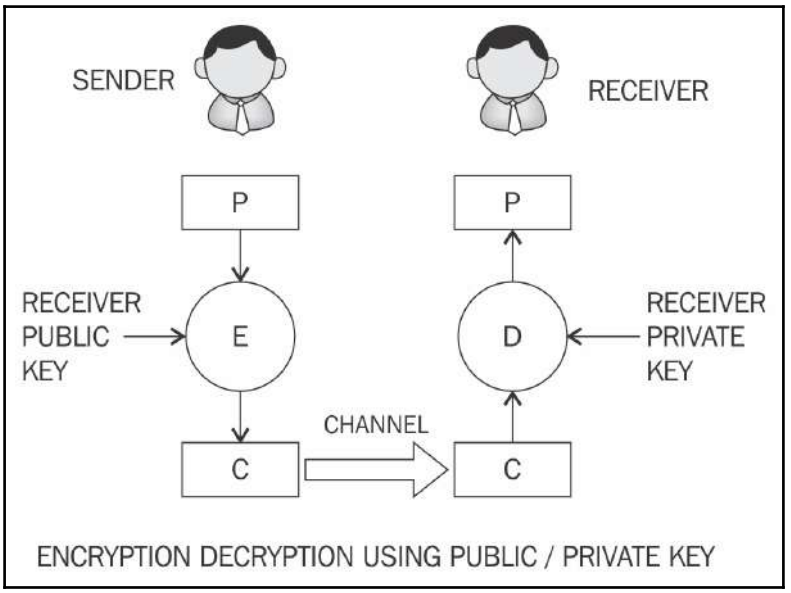
# Asymmetric cryptography

Asymmetric cryptography refers to a type of cryptography whereby the key that is used to encrypt the data is different from the key that is used to decrypt the data. Also known as public key cryptography, it uses public and private keys in order to encrypt and decrypt data, respectively. Various asymmetric cryptography schemes are in use, such as RSA, DSA, and El-Gammal.

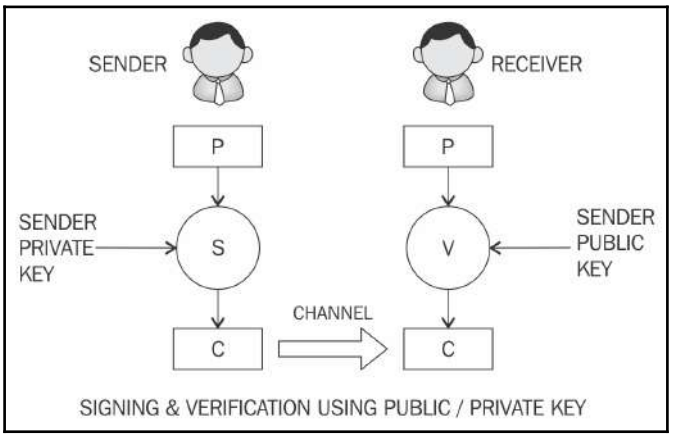
An overview of public key cryptography is shown in the following diagram:




Encryption decryption using public/private key

The diagram explains how a sender encrypts the data using a recipient's public key and is then transmitted over the network to the receiver. Once it reaches the receiver, it can be decrypted using the receiver's private key. This way, the private key remains on the receiver's side and there is no need to share keys in order to perform encryption and decryption, which is the case with symmetric encryption.

Another diagram shows how public key cryptography can be used to verify the integrity of the received message by the receiver. In this model, the sender signs the data using their private key and transmits the message across to the receiver. Once the message is received on the receiver's side, it can be verified for its integrity by the sender's public key. Note that there is no encryption being performed in this model. This model is only used for message authentication and validation purposes:




Model of a public key cryptography signature scheme

Security mechanisms offered by public key cryptosystem include key establishment, digital signatures, identification, encryption, and decryption.

Key establishment mechanisms are concerned with the design of protocols that allow setting up of keys over an insecure channel. Non-repudiation service, a very desirable property in many scenarios, can be provided using digital signatures. Sometimes, it is important to not only authenticate a user, but to also identify the entity involved in a transaction; this can also be achieved by a combination of digital signatures and challenge-response protocols. Finally, the encryption mechanism to provide confidentiality can also be realized using public key cryptosystems, such as RSA, ECC, or El-Gammal.

Public key algorithms are slower in computation as compared to symmetric key algorithms. Therefore, they are not commonly used in the encryption of large files or the actual data that needs encryption. They are usually used to exchange keys for symmetric algorithms and once the keys are established securely, symmetric key algorithms can be used to encrypt the data.

Public key cryptography algorithms are based on various underlying mathematical problems. There are three main families of asymmetric algorithms that are described here.

# Integer factorization

These schemes are based on the fact that large integers are very hard to factor. RSA is the prime example of this type of algorithm.

# Discrete logarithm

This is based on a problem in modular arithmetic that it is easy to calculate the result of modulo function but it is computationally infeasible to find the exponent of the generator. In other words, it is extremely difficult to find the input from the result. This is a one-way function.

For example, consider the following equation:

$$3^2 \bmod 10 = 9$$

Now given 9 finding 2, the exponent of the generator 3 is very hard. This hard problem is commonly used in **Diffie-Hellman** key exchange and digital signature algorithms.

# Elliptic curves

This is based on the discrete logarithm problem discussed earlier, but in the context of elliptic curves. Elliptic curve is an algebraic cubic curve over a field, which can be defined by an equation shown here. The curve is non-singular, which means that it has no cusps or self-intersections. It has two variables *a, b*, along with a point of infinity.

$$y^2 = x^3 + ax + b$$

Here, *a*, *b* are integers that can have various values and are elements of the field on which the elliptic curve is defined. Elliptic curves can be defined over reals, rational numbers, complex numbers, or finite fields. For cryptographic purposes, elliptic curve over prime finite fields is used instead of real numbers. Additionally, the prime should be greater than 3. Different curves can be generated by varying the value of *a*, *b.*

Mostly prominently used cryptosystems based on elliptic curves are **Elliptic Curve Digital Signatures Algorithm** (**ECDSA**) and **Elliptic Curve Diffie-Hellman** (**ECDH**) key exchange.

# Public and private keys

In order to understand public key cryptography, the first concept that needs to be looked at is the idea of public and private keys.

A private key, as the names suggests, is basically a randomly generated number that is kept secret and held privately by the users. Private key needs to be protected and no unauthorized access should be granted to that key; otherwise, the whole scheme of public key cryptography will be jeopardized as this is the key that is used to decrypt messages. Private keys can be of various lengths depending upon the type and class of algorithms used. For example, in RSA, typically, a key of 1024-bit or 2048-bits is used. 1024-bit key size is no longer considered secure and at least 2048 bit is recommended to be used in practice.

A public key is the public part of the private-public key pair. A public key is available publicly and published by the private key owner. Anyone who would then like to send the publisher of the public key an encrypted message can do so by encrypting the message using the published public key and sending it to the holder of the private key. No one else would be able to decrypt the message because the corresponding private key is held securely by the intended recipient. Once the public key encrypted message is received, the recipient can decrypt the message using the private key. There are a few concerns regarding public keys, such as authenticity and identification of the publisher of the public keys.

# RSA

A description of RSA is discussed here. RSA was invented in 1977 by *Ron Rivest*, *Adi Shamir*, and *Leonard Adelman*, hence the name RSA. This is based on the integer factorization problem, where the multiplication of two large prime numbers is easy but difficult to factor it back to the two original numbers.

The crux of the work in the RSA algorithm is during the key generation process. An RSA key pair is generated by performing the steps described here.

Modulus generation:

- Select $p$ and $q$ very large primes
- Multiply $p$ and $q$ , $n=p.q$ to generate modulus $n$

Generate co-prime:

- Assume a number called $e$.

- It should satisfy certain conditions, that is, it should be greater than 1 and less than *(p-1) (q-1)*. In other words, e must be such a number that no number other than 1 can be divided into *e* and *(p-1) (q-1)*. This is called co-prime, that is, *e* is the co-prime of *(p-1)(q-1)*.

Generate public key:

- Modulus generated in step 1 and e generated in step 2 is pair that, together, is a public key. This part is the public part that can be shared with anyone; however, *p* and *q* need to be kept secret.

Generate private key:

- Private key called d here and is calculated from *p*, *q* and *e*. Private key is basically the inverse of e modulo *(p-1)(q-1)*. In the equation form, it is this:

$$ed = 1 \bmod (p-1)(q-1)$$

Usually, an extended Euclidean algorithm is used to calculate *d*; this algorithm takes *p*, *q* and *e* and calculates *d*. The key idea in this scheme is that anyone who knows *p* and *q* can calculate private key *d* easily, by applying the extended Euclidean algorithm, but someone who doesn't know the value of *p* and *q* cannot generate *d*. This also implies that *p* and *q* should be large enough for the modulus *n* to become very difficult (computationally infeasible) to factor.

# Encryption and decryption using RSA

RSA uses the following equation to produce cipher text:

$$C = P^e \bmod n$$

This means that plain text *P* is raised to *e* number of times and then reduced to modulo *n*.

Decryption in RSA is given by the following equation:

$$P = C^d \bmod n$$

This means that the receiver who has a public key pair (*n*, *e*) can decipher the data by raising *C* to the value of the private key *d* and reducing to modulo *n*.

# Elliptic curve cryptography (ECC)

ECC is based on the discrete logarithm problem that is based on elliptic curves over finite fields (Galois fields). The main benefit of ECC over other types of public key algorithms is that it needs a smaller key size while providing the same level of security as, for example, RSA. Two notable schemes that originate from ECC are **Elliptic Curve Diffie-Hellman** (**ECDH**) for key exchange and **Elliptic Curve Digital Signature Algorithm** (**ECDSA**) for digital signatures. It can also be used for encryption but is not usually used for this purpose in practice; instead, key exchange and digital signatures are more commonly used. As ECC needs less space to operate, it is becoming very popular on embedded platforms or in systems where storage resources are limited. As a comparison, the same level of security can be achieved in ECC by only using 256-bit operands as compared to 3072-bits in RSA.

## Mathematics behind ECC

In order to understand ECC, a basic introduction to the underlying mathematics is necessary. Elliptic curve is basically a type of polynomial equation known as weierstrass equation that generates a curve over a finite field. The most commonly used field is where all arithmetic operations are performed modulo *a* prime *p*. Elliptic curve groups consist of points on the curve over a finite field.

An elliptic curve can be defined as an equation here:

$$y^2 = x^3 + Ax + B \bmod p$$

Here, *A* and *B* belong to a finite field *Zp* or *FP* (prime finite field) along with a special value called point of infinity. Point of infinity $\infty$ is used to provide identity operations for points on the curve.

Furthermore, a condition also needs to be met that ensures that the equation mentioned earlier has no repeated roots. This means that the curve is non-singular.

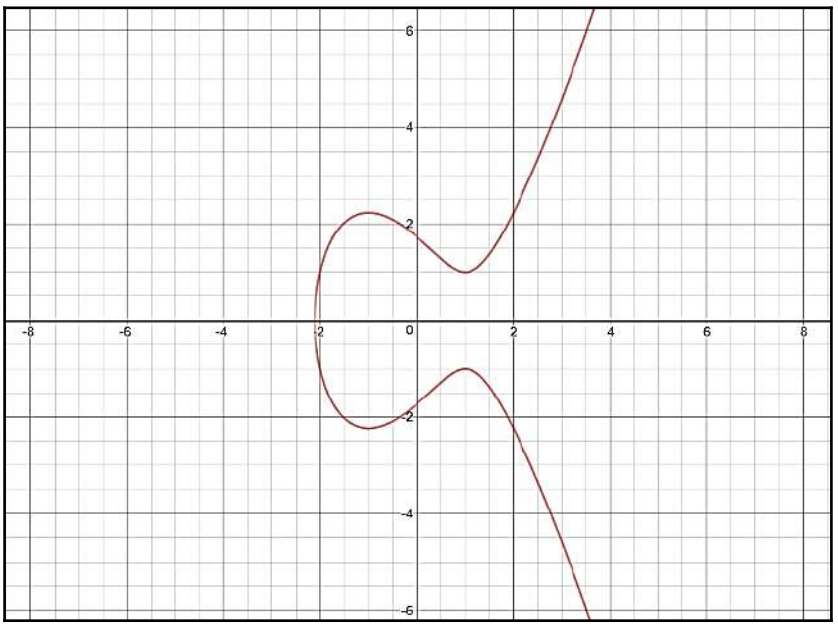
The condition is described here in the equation, which is a standard requirement that needs to be met. More precisely, this ensures that the curve is nonsingular:

$$4a^3 + 27b^2 \neq 0 \bmod p$$

A real number representation of elliptic curve can be visualized as shown in the following graph. This is a graph of equation over real numbers:

$$y^2 = x^3 + ax + b$$

The actual curves used in elliptic curve cryptography are over finite prime fields, but here, they are shown over real number as it becomes easier to visualize the operations when graphed over *R*:



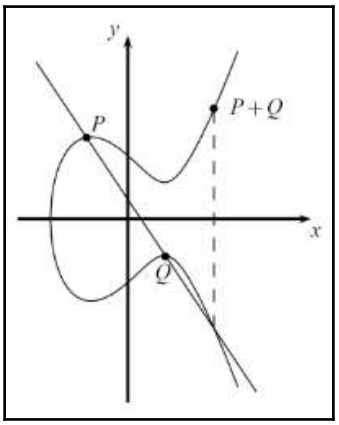

Elliptic curve over reals, a = -3 and b = 3

In order to construct the discrete logarithm problem based on elliptic curves, a large enough cyclic group is required. First, the group elements are identified as a set of points that satisfy the earlier equation. After this, group operations need to be defined on these points.

Group operations on elliptic curves are point addition and point doubling. Point addition is a process where two different points are added and point doubling means that the same point is added to itself. Both of these operations can be visualized as shown in the following diagrams.

## Point addition

Point addition is shown in the following diagram. This is a geometric representation of point addition on elliptic curves. In this method, a line is drawn through the curve that intersects the curve at two points shown below *P* and *Q*, which yields a third point between the curve and the line. This point is mirrored as *P*+*Q*, which represent the result of addition as *R*. This is shown as *P*+*Q* in the following diagram:




Point addition visualized over R

Group operation denoted by sign + for addition yields the following equation:

$$P + Q = R$$

In this case, two points are added in order to compute the coordinates of the third point on the curve:

$$P + Q = R$$

More precisely, this means that coordinates are added as shown in the following equation:

$$(x1, y1) + (x2, y2) = (x3, y3)$$

The equation of point addition is as follows:

$$x_3 = s^2 - x_1 - x^2 \bmod p$$

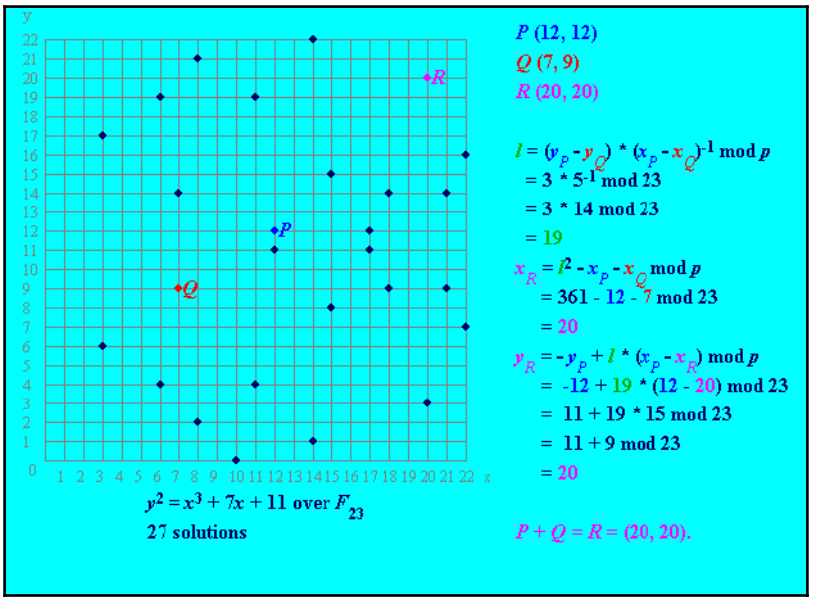
$$y_3 = s(x_1 - x_3) - y_1 \bmod p$$

Here, this is the result:

$$S = \frac{(y_2 - y_1)}{(x_2 - x_1)} \bmod p$$

*S* in the preceding equation depicts the line going through *P* and *Q*.

An example of point addition shown here is produced using Certicom's online calculator. This example shows the addition and solutions for the equation over finite field $F_{23}$. This is in contrast to the example shown earlier, which is over real numbers and only shows the curve but no solutions to the equation:




Example of point addition using Certicom's online calculator tool

In the example, the graph on the left-hand side shows the points that satisfy the equation shown here:

$$y^2 = x^3 + 7x + 11$$

There are 27 solutions to the equation shown earlier over a finite field $F_{23}$. *P* and *Q* are chosen to be added to produce the point *R*. Calculations are shown on the right-hand side, which calculates the third point *R*. Note that here, $l$ is used to depict the line going through *P* and *Q*.

As an example to show how the equation is satisfied by the points shown in the graph, a point (*x*, *y*) is picked up where $x = 3$ and $y = 6$.

Using these values in the equation shows that the equation is satisfied indeed. This is shown as follows:

$$y^2 mod\ 23 = x^3 + 7x + 11\ mod\ 23$$

$$6^2 mod\ 23 = 3^3 + 7(3) + 11\ mod\ 23$$

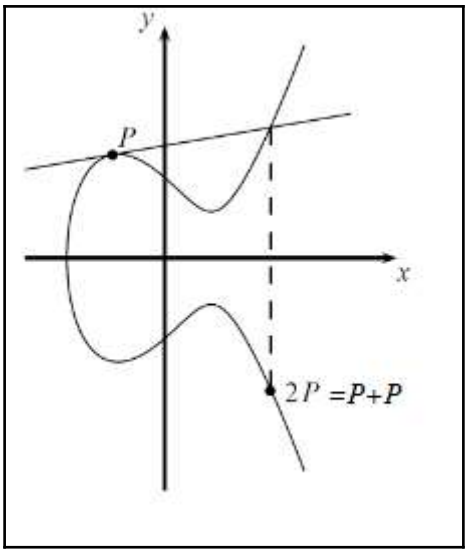
$$36\ mod\ 23 = 59\ mod\ 23$$

$$13 = 13$$

The next section will introduce the concept of point doubling, which is another operation that can be performed on elliptic curves.

## Point doubling

The other group operation on elliptic curves is called point doubling and is described in the following diagram. This is a process where *P* is added into itself. In this method, a tangent line is drawn through the curve, as shown in the following graph. The second point is obtained, which is at the intersection of the tangent line drawn and the curve. This point is then mirrored to yield the result, which is shown as $2P = P + P$:




Graph representing point doubling over real numbers

In case of point doubling, the equation becomes as follows:

$$x_3 = s^2 - x_1 - x^2 \bmod p$$

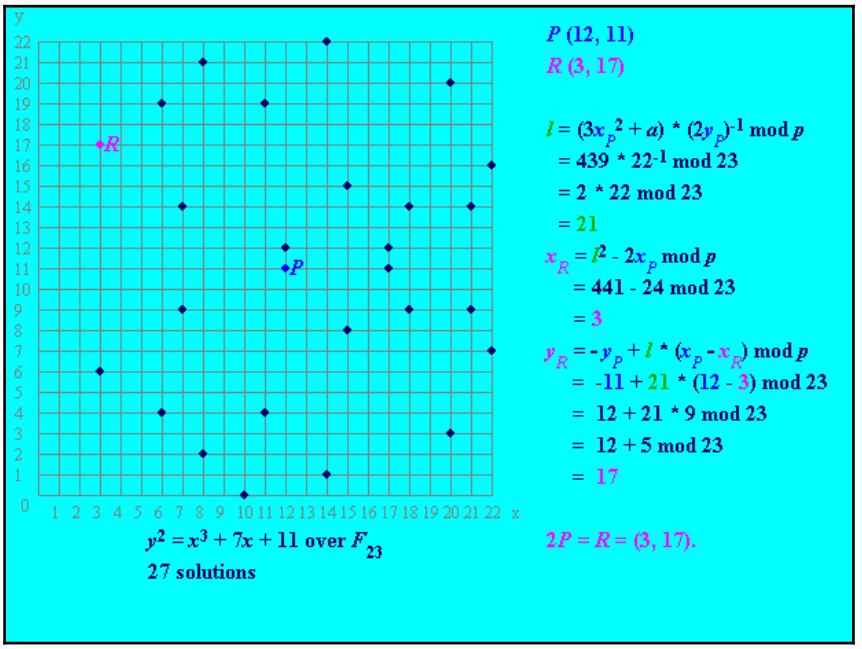
$$y_3 = s(x_1 - x_3) - y_1 \bmod p$$

$$S = \frac{(y_2 - y_1)}{(x_2 - x_1)} \bmod p$$

Here, *S* is the slope of tangent (tangent line) going through *P*. It is the line on top shown in the preceding figure. In the preceding example, the curve is plotted over reals as a simple example and no solution to the equation is shown.

An example is shown here, which shows the solutions and point doubling of elliptic curve over finite field $F_{23}$. The graph on the left-hand side shows the points that satisfy the equation:

$$y^2 = x^3 + 7x + 11$$




Example of point doubling using certicom's online calculator tool

As shown earlier, on the right-hand side, a calculation is shown that finds the *R* after *P* is added into itself (point doubling). There is no *Q* as here, the same point *P* is used for doubling. Note that in the calculation, *l* is used to depict the tangent line going through *P*.

In the next section, an introduction to the discrete logarithm problem will be presented.

# Discrete logarithm problem

The discrete logarithm problem in ECC is based on the idea that under certain conditions, all points on an elliptic curve form a cyclic group.
On an elliptic curve, the public key is a random multiple of the generator point, whereas the private key is a randomly chosen integer used to generate the multiple. In other words, a private key is a randomly chosen integer, whereas the public key is a point on the curve. The discrete logarithm problem is used to find the private key (an integer) where that integer falls within all points on the elliptic curve. An upcoming equation shows this precisely.
Consider an elliptic curve *E*, with two elements *P* and *T*. The discrete logarithmic problem is to find the integer d, where *1 <= d <= #E*, such that:

$$P + P + \ldots + P = d\,P = T$$

Here, *T* is the public key (point on the curve) and d is the private key. In other words, public key is a random multiple of generator, whereas the private key is the integer that is used to generate the multiple. *#E* represents the order of the elliptic curve, which basically means the number of points that are present in the cyclic group of the elliptic curve. A cyclic group is formed by a combination of points on the elliptic curve and point at infinity.

A key pair is linked with specific domain parameters of an elliptic curve. Domain parameters include a field size, field representation, two elements from the field *a* and *b*, two field elements *Xg* and *Yg*, order n of point *G* that is calculated as *G=(Xg, Yg)* and the co-factor *h = #E(Fq)/n*. A practical example using OpenSSL will be described later in this section.

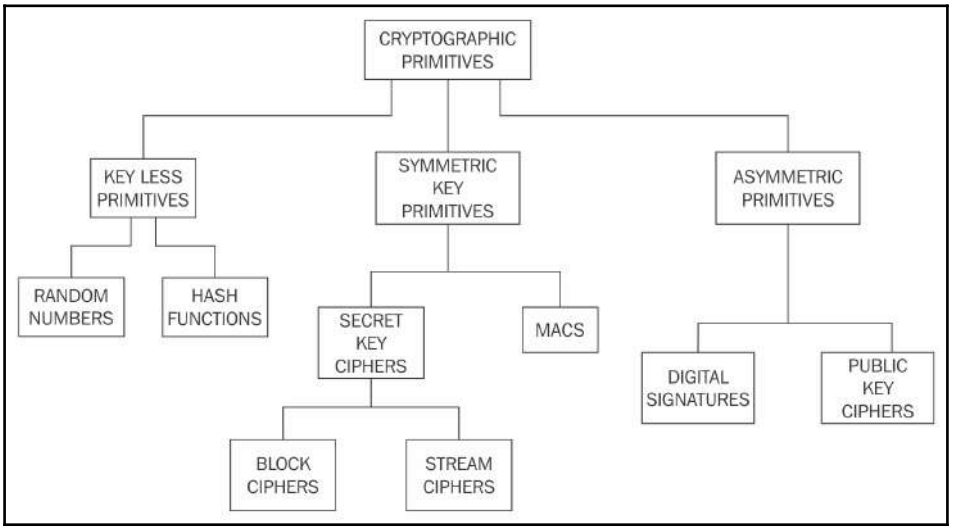
```
      36:41:41
Cofactor:  1 (0x1)
```

The preceding example shows the prime number used and values of `A` and `B` the with generator, order, and cofactor of the `SECP256K1` curve domain parameters.

There is another category of cryptographic primitives that is known as hash functions. Hash functions are not used to encrypt; data instead, they produce a fixed length digest of text.

# Cryptographic primitives

This taxonomy of cryptographic primitives can be visualized as shown here:




Cryptographic primitives

# Hash functions

Hash functions are used to create fixed length digests of arbitrarily long input strings. Hash functions are keyless and provide the data integrity service. They are usually built using iterated and dedicated hash function construction techniques. Various families of hash functions are available, such as MD, SHA1, SHA-2, SHA-3, RIPEMD, and Whirlpool. Hash functions are commonly used in digital signatures and message authentication codes, such as HMACs. They have three security properties, namely pre-image resistance, second pre-image resistance, and collision resistance. These properties are explained later in the section.

**Hash functions** are typically used to provide data integrity services. These can be used as one-way functions and to construct other cryptographic primitives, such as MACs and digital signatures. Some applications used hash functions as a means of generating **pseudo random numbers** (**PRNGs**). Hash functions do not require a key. There are two practical and three security properties of hash functions that must be met depending on the level of requirements of integrity.

## Compression of arbitrary messages into fixed length digest

This property is concerned with the fact that a hash function must be able to take a long input text of any length and output a fixed length compressed message. Hash functions produce a compressed output in various bit sizes, usually between 128-bits and 512-bits.

## Easy to compute

Hash functions are efficient and fast one-way functions. The requirement is that they be very quick to compute regardless of the message size. The efficiency may decrease if the message is too big but the function should still be fast enough for practical use.

In the following section, security properties of hash functions are discussed.

## Pre-image resistance

Consider an equation:

$$h(x) = y$$

Here, *h* is the hash function, *x* is the input, and *y* is the hash. The first security property requires that *y* cannot be reverse computed to *x*. *x* is considered a *pre-image* of *y*, hence the name pre-image resistance. This is also called one-way property.

## Second pre-image resistance

This property requires that given *x* and *h(x)* , it is almost impossible to find any other message *m* , where *m != x* and *hash of m = hash of x*. *h(m) = h(x)*. This property is also known as weak collision resistance.

## Collision resistance

This property requires that two different input messages should not hash to the same output. In other words, *h(x) != h(z)*. This property is also known as strong collision resistance.

Hash functions, due to their very nature, will always have some collisions, and that is where two different messages hash to the same output, but they should be computationally infeasible to find. A concept known as **avalanche effect** is desirable in all hash functions. Avalanche effect specifies that a small change, even a single character change in the input text, will result in a totally different hash output.

Hash functions are usually designed by following iterated hash functions approach. In this method, the input message is compressed in multiple rounds on a block-by-block basis to produce the compressed output. A popular type of iterated hash function is Merkle-Damgard construction. This construction is based on the idea of dividing the input data into equal sizes of blocks and then feeding them through the compression functions in an iterative manner. The collision resistance of the property of compression functions ensures that the hash output is also collision-resistant. Compression functions can be built using block ciphers. In addition to Merkle-Damgard, there are various other constructions of compression functions proposed by researchers, for example, *Miyaguchi-Preneel* and *Davies-Meyer*.

There are multiple hash function categories. You will be introduced to these categories in the upcoming section.

## Message Digest (MD)

Message Digest functions were very popular in early 1990s. MD4 and MD5 are members of this category. Both MD functions are found to be insecure and not recommended for use any more. MD5 is a 128-bit hash function that was commonly used for file integrity checks.

## Secure Hash Algorithms (SHAs)

**SHA-0**: This is a 160-bit function introduced by NIST in 1993.

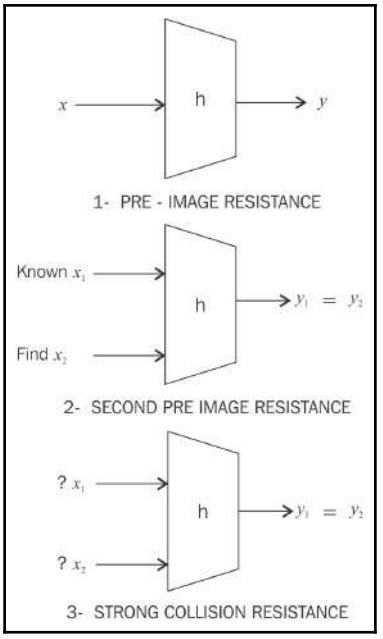
**SHA-1**: SHA-1 was introduced later by NIST as a replacement of SHA-0. This is also a 160-bit hash function. SHA-1 is used commonly in SSL and TLS implementations. It should be noted that SHA-1 is now considered insecure and is being deprecated by certificate authorities. Its usage is now discouraged in any new implementations.

**SHA-2**: This category includes four functions defined by the number of bits of the hash: SHA-224, SHA-256, SHA-384 and SHA-512.

**SHA-3**: This is the latest family of SHA functions. SHA3-224, SHA3-256, SHA3-384 and SHA3-512 are members of this family. SHA3 is a NIST-standardized version of Keccak. Keccak uses a new approach called *sponge construction* instead of the commonly used Merkle-Damgard transformation.

**RIPEMD**: RIPEMD is the acronym for *RACE Integrity Primitives Evaluation Message Digest*. It is based on the design ideas used to build MD4. There are multiple versions of RIPEMD, including 128-bit, 160-bit, 256-bit, and 320-bit.

**Whirlpool**: This is based on a modified version of Rijndael cipher known as W. It uses the Miyaguchi-Preneel compression function, which is a type of one-way function used for the compression of two fixed length inputs into a single fixed length output. It is a single block length compression function:




Three security properties of hash functions

Hash functions have many practical applications ranging from simple file integrity checks and password storage to be used in cryptographic protocols and algorithms. They are used in hash tables, distributed hash tables, bloom filters, virus finger printing, peer-to-peer P2P file sharing, and many other applications.

In blockchain, hash functions play a very vital role. Especially, the proof of work function uses SHA-256 twice in order to verify the computational effort spent by miners. RIPEMD 160 is used to produce bitcoin addresses. This will be discussed in more detail in later chapters.

## Design of Secure Hash Algorithms (SHA)

In the following section, you will be introduced to the design of SHA-256 and SHA-3. Both of these are used in bitcoin and Ethereum, respectively. Ethereum doesn't use NIST Standard SHA-3 but Keccak, which is the original algorithm presented to NIST. NIST, after some modifications such as increase in the number of rounds and simpler message padding, standardized Keccak as SHA-3.

## SHA-256

SHA-256 has the input message size < 2^64-bits. Block size is 512-bits and has a word size of 32-bits. Output is 256-bit digest.

The compression function processes a 512-bit message block and a 256-bit intermediate hash value. There are two main components of this function: compression function and a message schedule.

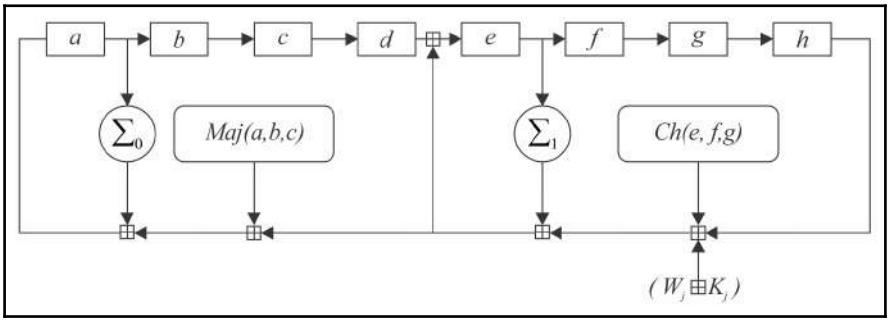
The algorithm works as follows:

- Pre-processing:

1. Padding of the message, which is used to make the length of a block to 512-bits if it is smaller than the required block size of 512-bits.
2. Parsing the message into message blocks that ensure that the message and its padding is divided into equal blocks of 512-bits.
3. Setting up the initial hash value, which is the eight 32-bit words obtained by taking the first 32-bits of the fractional parts of the square roots of the first eight prime numbers. These initial values are randomly chosen in order to initialize the process and gives a level of confidence that no backdoor exists in the algorithm.

- Hash computation:

1. Each message block is processed in a sequence and requires 64 rounds to compute the full hash output. Each round uses slightly different constants to ensure that no two rounds are the same.
2. First, the message schedule is prepared.
3. Then, eight working variables are initialized.
4. Then, the intermediate hash value is calculated.
5. Finally, the message is processed and the output hash is produced:



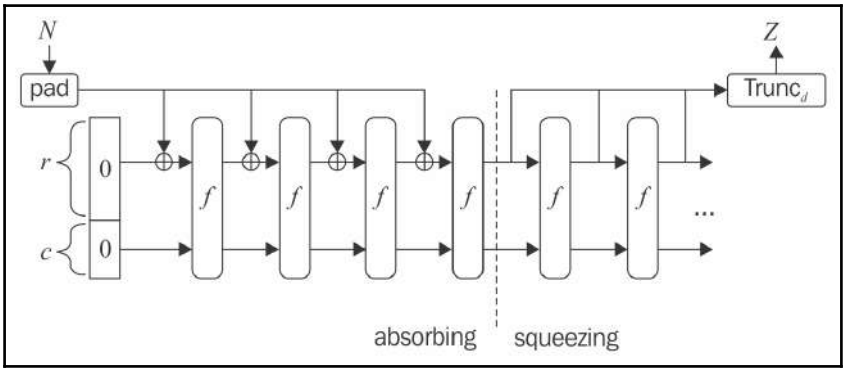

one round of SHA 256 compression function

In the preceding diagram, *a*, *b*, *c*, *d*, *e*, *f*, *g*, and *h* are the registers. **Maj** and **Ch** are applied bitwise. $\Sigma_0$ and $\Sigma_1$ performs bitwise rotation. Round constants are $\mathbf{W}_j$ and $\mathbf{K}_j$ , which are added *mod 2^32*.

## Design of SHA3 (Keccak)

The structure of SHA-3 is very different from the usual SHA-1 and SHA-2. The key idea behind SHA-3 is based on un-keyed permutations as opposed to other usual hash functions' constructions that used keyed permutations. Keccak also does not make use of the Merkle-Damgard transformation that is commonly used to handle arbitrary length input messages in hash functions. A newer approach called sponge and squeeze construction is used in Keccak, which is basically a random permutation model. Different variants of SHA3 have been standardized, such as SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128, and SHAKE256. SHAKE128 and SHAKE256 are extendable output functions that are also standardized by `NIST.XOF` functions that allow the output to be extended to any desired length.

The following diagram shows the sponge and squeeze model that is the basis of SHA3 or Keccak. As an analogy to sponge, first, the data is absorbed into the sponge after applying padding, where it is then changed into a subset of permutation state using XOR and then the output is squeezed out of the sponge function that represents the transformed state. Rate is the input block size of a sponge function, whereas capacity determines the generic security level:




SHA-3 absorbing and squeezing function in SHA3

## OpenSSL example of hash functions

The following command will produce a hash of 256-bits of Hello messages using the SHA256 algorithm:

```
:~/Crypt$ echo -n 'Hello' | openssl dgst -sha256
(stdin)= 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
```

Note that even a small change in the text, such as changing the case of *H*, results in a big change in the output hash. This is known as *avalanche effect*, as discussed earlier:

```
:~/Crypt$ echo -n 'hello' | openssl dgst -sha256
(stdin)= 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
```

Note that both outputs are completely different:

```
Hello:
18:5f:8d:b3:22:71:fe:25:f5:61:a6:fc:93:8b:2e:26:43:06:ec:30:4e:da:51:80:07:
d1:76:48:26:38:19:69
hello:
2c:f2:4d:ba:5f:b0:a3:0e:26:e8:3b:2a:c5:b9:e2:9e:1b:16:1e:5c:1f:a7:42:5e:73:
04:33:62:93:8b:98:24
```

## Message Authentication codes (MACs)

MACs are sometimes called keyed hash functions and can be used to provide message integrity and authentication. In others words, they are used to provide data origin authentication. These are symmetric cryptographic primitives using a shared key between the sender and the receiver. MACs can be constructed using block ciphers or hash functions.

## MACs using block ciphers

In this approach, block ciphers are used in the **Cipher block chaining mode** (**CBC mode**) in order to generate a MAC. Any block cipher-for example, AES in the CBC mode-can be used. The MAC of the message is in fact the output of the last round of the CBC operation. The length of the MAC output is the same as the block length of the block cipher used to generate MAC. MACs are verified simply by computing the MAC of the message and comparing it with the received MAC. If they are the same, then the message integrity is confirmed; otherwise, the message is considered altered. It should also be noted that MACs work like digital signatures, but they cannot provide the nonrepudiation service due to their symmetric nature.

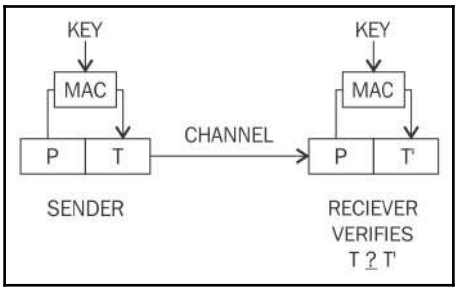
## HMACs (hash-based MACs)

Similar to the hash function, they produce a fixed length output and take an arbitrarily long message as the input. In this scheme, the sender signs a message using MAC and the receiver verifies it using the shared key. The key is hashed with the message using either of the two methods known as secret prefix or the secret suffix method. In the first method, the key is concatenated with the message, that is, the key comes first and the message comes after, whereas in the latter method, the key comes after the message:

$$\text{Secret prefix: } M = MACk(x) = h(k||x)$$

$$\text{Secret suffix: } M=MACk(x) = h(x||k)$$

There are pros and cons of both methods. Some attacks on both schemes have been discovered. There are HMAC constructions schemes that use various techniques, such as **ipad** and **opad** (inner padding and outer padding) proposed by researchers that are considered secure with some assumptions:

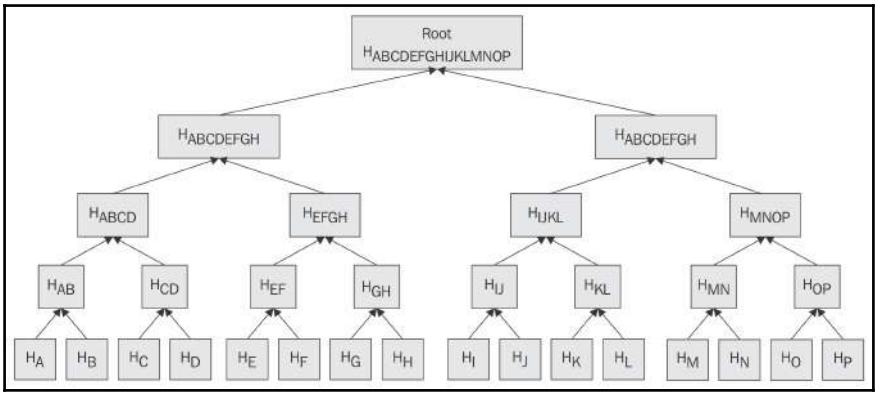



Operation of a MAC function

# Merkle trees

The concept of Merkle tree was introduced by *Ralph Merkle*. A visualization of Merkle tree is shown here, which makes it easy to understand. Merkle trees allow secure and efficient verification of large data sets.

It is a binary tree in which first, the inputs are placed at the leaves (node with no children), and then values of pairs of child nodes are hashed together in order to produce a value for the parent node (internal node) until a single hash value known as Merkle root is achieved:




A Merkle tree

# Patricia trees

In order to understand Patricia trees, first, you will be introduced to the concept of a **trie**. A trie or a digital tree is an ordered tree data structure used to store a dataset.

**Practical Algorithm to Retrieve Information Coded in Alphanumeric** (**Patricia**), also known as Radix tree, is a compact representation of a trie in which a node that is the only child of a parent is merged with its parent.

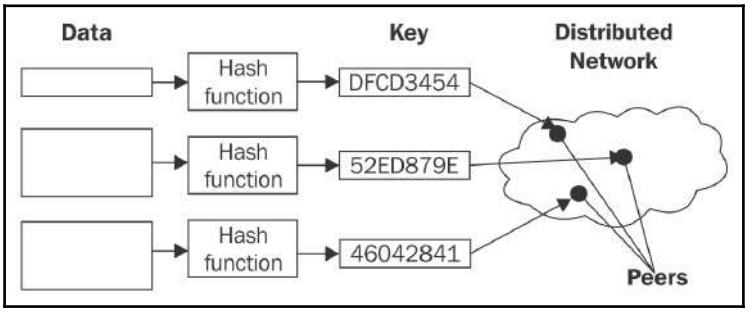
**Merkle-Patricia tree**, based on the definitions of Patricia and Merkle, is a tree that has a root node that contains the hash value of the entire data structure.

# Distributed hash tables (DHTs)

A hash table is a data structure that is used to map keys to values. Internally, a hash function is used to calculate an index into an array of buckets, from which the required value can be found. Buckets have records stored in them using a hash key and are organized in a particular order.

With the definition provided earlier in mind, one can think of the distributed hash table as a data structure where data is spread across various nodes and nodes are equivalent to buckets in a peer-to-peer to network.

The following diagram visually shows how a DHT works. The example shows that data is passed through a hash function, which results in generating a compact key. This key is then linked with the data (values) on the peer-to-peer network. When users on the network request the data (via the filename), the filename can be hashed again to produce the same key and any node on the network can then be requested to find the corresponding data. DHTs provides decentralization, fault tolerance, and scalability:




Distributed hash tables

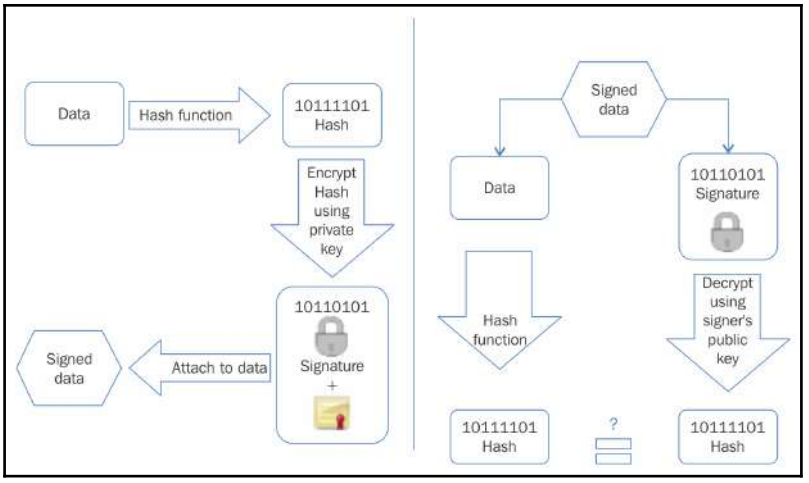
# Digital signatures

Digital signatures provide a means of associating a message with an entity from which the message has been originated. Digital signatures are used to provide data origin authentication and nonrepudiation. They are calculated in two steps. High-level steps of an RSA digital signature scheme is given as follows:

1. Calculate the hash value of the data packet. This will provide the data integrity guarantee as hash can be computed at the receiver's end again and matched with the original hash to check whether the data has been modified in transit. Technically, message signing can work without hashing the data first, but is not considered secure.
2. The second step signs the hash value with the signer's private key. As only the singer has the private key, the authenticity of the signature and the signed data is ensured.

Digital signatures have some important properties, such as authenticity, unforgeability, and nonreusability. Authenticity means that the digital signatures are verifiable by a receiving party. The unforgeability property ensures that only the sender of the message is able to use the signing functionality using the private key. In other words, no one else should be able to produce the signed message that has been produced by the legitimate sender. Non reusability means that the digital signature cannot be separated from a message and used for another message again.

The operation of a generic digital signature function is shown in the following diagram:




Digital signing (left) and verification process (right) (Example of RSA digital signatures)

If a sender wants to send an authenticated message to a receiver, there are two methods that can be used. These two approaches to use digital signatures with encryption are introduced here.

### Sign then encrypt

In this approach, the sender digitally signs the data using the private key, appends the signature to the data, and then encrypts the data and the digital signature using the receiver's public key. This is considered a more secure scheme as compared to the encrypt then sign scheme described next.

### Encrypt then sign

In this approach, the sender encrypts the data using the receiver's public key and then digitally signs the encrypted data.

In practice, a digital certificate that contains the digital signature is issued by a **certificate authority** (**CA**) that associates a public key with an identity.

Various schemes, such as RSA, Digital Signature Algorithm, and Elliptic Curve Digital Signature Algorithm-based digital signature schemes are used in practice. RSA is the most commonly used; however, with the traction of elliptic curve cryptography, ECDSA-based schemes are also becoming quite popular.

The ECDSA scheme is described in detail here.

# Elliptic Curve Digital signature algorithm (ECDSA)

In order to sign and verify using the ECDSA scheme, the first key pair needs to be generated:

1. First, define an elliptic curve *E*:
    1. With modulus *P*.
    2. Coefficients *a* and *b*.
    3. Generator point *A* that forms a cyclic group of prime order *q*.